

Week 15 : Automation 、 Safety 、 Fault Recovery與 Clean Reconstruction

日期：2026-12-16

本章將KY-018輸入、RGB輸出、狀態機、網路、Backend、Database與手機前台組成一項可說明 的自動反應。學生先寫清楚trigger、允許條件、停止條件與優先順序，再加入hysteresis、連續取樣、最大動作時間及斷線策略；最後以故障注入驗證安全復原，並由另一人從乾淨 資料夾依文件重建系統。

光線觸發 RGB，STOP 可中止自動反應

本週讓光線觸發 RGB 變色，再確認 STOP 與故障處理仍能停止自動反應。只使用已確認的按鈕、KY-018 與 RGB，不新增馬達或舵機。

1. 保留上次成功的 Week 12 個人程式。另開 [week15_automation_device.ino](#)，在 IDE 另存新檔為 `week15_automation_practice`。
2. 依 Week 12 的方式建立 `secrets.h`，填本次 Wi-Fi 與 broker 帳密；保持 `DRY_RUN=true`。個人機密不放入教材或提交 Git。
3. 用 Ctrl+F 逐一核對：`DEVICE_ID`、`PIN_START`、`PIN_STOP`、`PIN_LIGHT`、`PIN_RGB_R`、`PIN_RGB_G`、`PIN_RGB_B`、`RGB_ON_LEVEL`、`LIGHT_VALID_MIN`、`LIGHT_VALID_MAX`。這些值來自已成功的 Week 12 設定，不因換檔案就自動帶過來。
4. 再找 `DARK_WHEN_RAW_LESS`、`DARK_ENTER_RAW`、`LIGHT_EXIT_RAW`。依第二、三節及本人的有效光線紀錄設定；沒有校正資料時只能先編譯，不猜門檻上電。
5. 依 [Week 12 啟動步驟](#) 啟動 broker、後端與 bridge；已有密碼檔不要重建。A、D、E 都保持執行。
6. 第五節的七段修改已包含在完整程式中，**不要再貼一次**。編譯、接線確認及操作步驟見第六節。

本週 `auto_on` 等命令不在手機預設選單；[下面有逐步送出方式](#)，透過後端 API 建立命令，沿用相同權限與 `command_id` 追蹤。

一、本週內容

教學目標

完成本單元後，學生應能：

1. 訂定自動化規則 (automation policy)，明列可觀察的觸發條件 (trigger)、資料有效性要求 (validity requirement)、狀態前置條件 (state precondition)、動作 (action)、最長持續時間 (maximum duration)、停止條件 (stop condition) 與安全狀態 (safe state)。
2. 實作感測遲滯 (hysteresis)、重複樣本確認 (repeated-sample confirmation)、狀態轉換 (state transition)、明確的手動 / 自動控制權 (manual/automatic ownership)、實體停止優先權 (physical stop priority) 及具有明確限制的復原行為 (recovery)。
3. 依文件記錄的優先順序 (priority order)，處理自動動作、遠端命令 (remote command)、本機控制 (local control)、無效感測 (invalid sensing)、網路中斷 (network loss) 與動作逾時 (action timeout) 之間的衝突。
4. 注入並定位至少三種故障 (fault)，同時維持安全的實體輸出、可見的手機狀態、結構化紀錄 (structured log) 與可重現的復原步驟。
5. 從乾淨目錄重建後端 (backend)、資料庫結構 (database schema)、行動前端 (mobile frontend) 與裝置設定 (device configuration)，不依賴複製既有環境或未記錄的機密資料。
6. 區分主機測試 (host test)、編譯 (compilation)、上傳 (upload)、實體目標板測試 (physical target test)、故障測試 (fault test) 與重建證據 (reconstruction evidence)。

教學內容

本單元將自動化 (automation) 與全端物聯網系統 (full-stack IoT system) 所需的安全及復原能力整合。學生會將已校正的感測輸入轉為具有明確限制的狀態機動作 (state-machine action)，以遲滯 (hysteresis) 與重複取樣 (repeated sampling) 避免門檻附近反覆切換 (threshold chatter)，並明定實體停止 (physical stop)、故障 (fault)、手動命令 (manual command) 及自動行為的優先順序。實作會刻意引入網路及感測故障，同時查驗安全輸出、操作者回饋、資料庫紀錄 (database record) 與紀錄檔 (log)。最後進行乾淨環境重建 (clean reconstruction)，檢查文件中的軟體版本、環境變數 (environment variable)、資料庫初始化 (database initialization)、前端資源 (frontend asset)、裝置設定檔 (device profile) 與機密佔位值 (secret placeholder)，是否足以讓另一個人重現系統。

本週器材外觀

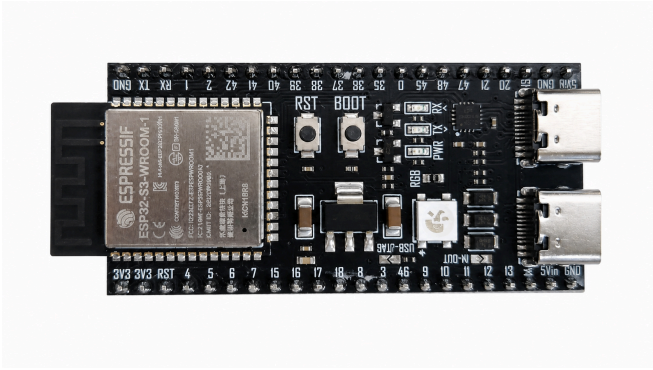
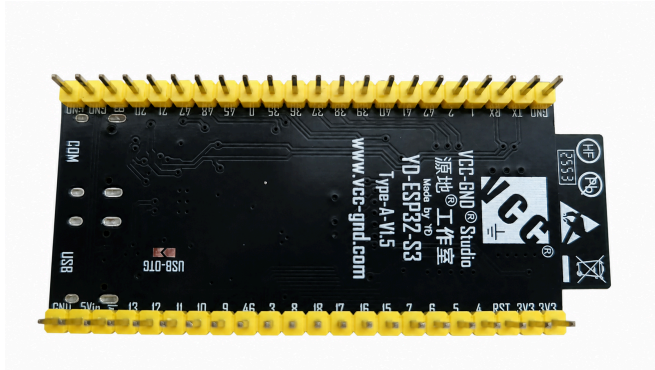
用光敏資料觸發 RGB 自動反應，測試 START / STOP、故障復原與重建；不新增硬體。另備筆電、USB 資料線、區域網路與手機。

照片下方標示拍攝角度與來源。先辨認零件，再依本週器材表與接線步驟操作；照片本身不是接線指令，也不表示已完成電氣驗證。

[ESP32-S3 開發板](#) · [400 孔麵包板](#) · [杜邦線](#) · [四腳輕觸按鈕](#) · [KY-018 光敏電阻模組](#) · [HW-479 三色發光二極體模組](#)

ESP32-S3 開發板 (Development Board)

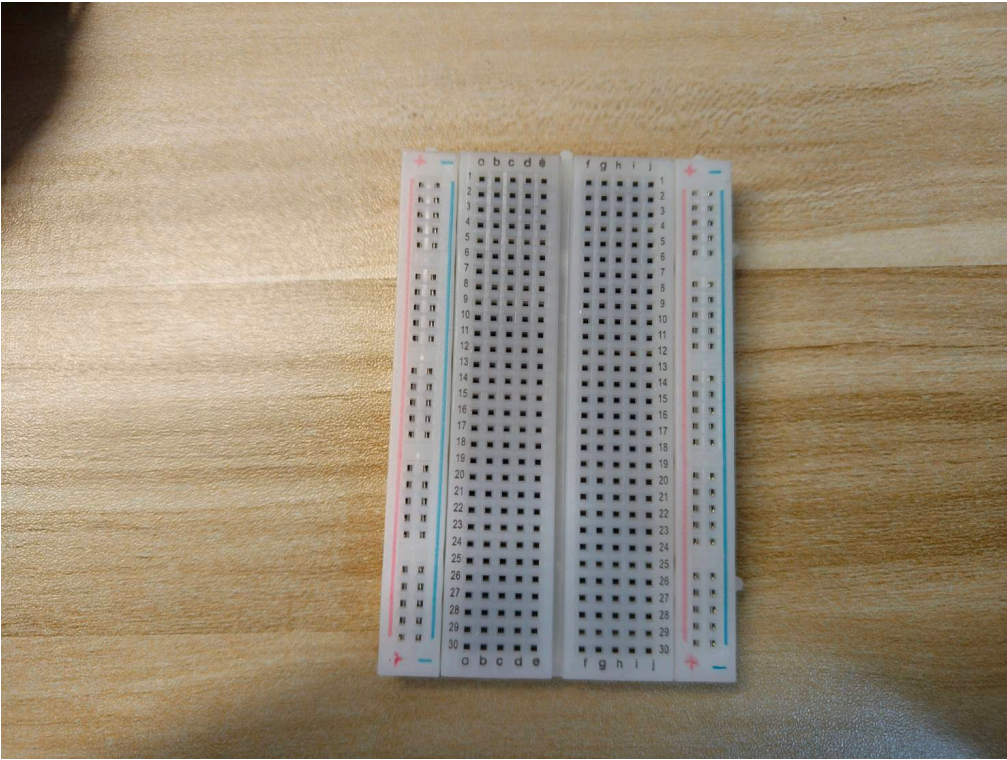
照片中的板卡為 YD-ESP32-S3 Type-A V1.5，搭載 N16R8 模組。正反面白底圖是既有後製展示圖；小字與腳位須核對本人實物及本週接線資料。

實物後製展示圖：正面：模組、按鈕與 USB 接頭	實物後製展示圖：背面：板身與排針
	

其他留存角度：[麵包板對孔紀錄](#)；不是建議的實驗安裝方式，右側接線空間不足。

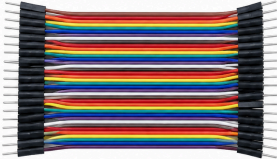
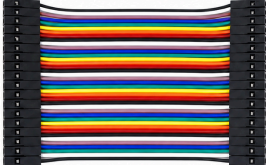
400 孔麵包板 (Breadboard)

辨認中央溝槽、a~j 字母與列號。外觀照片不表示所有孔都相通，連通關係依本週圖解與斷電量測確認。

實物照片：俯視：中央溝槽、五孔組與側邊電源軌


杜邦線 (Jumper Wire)

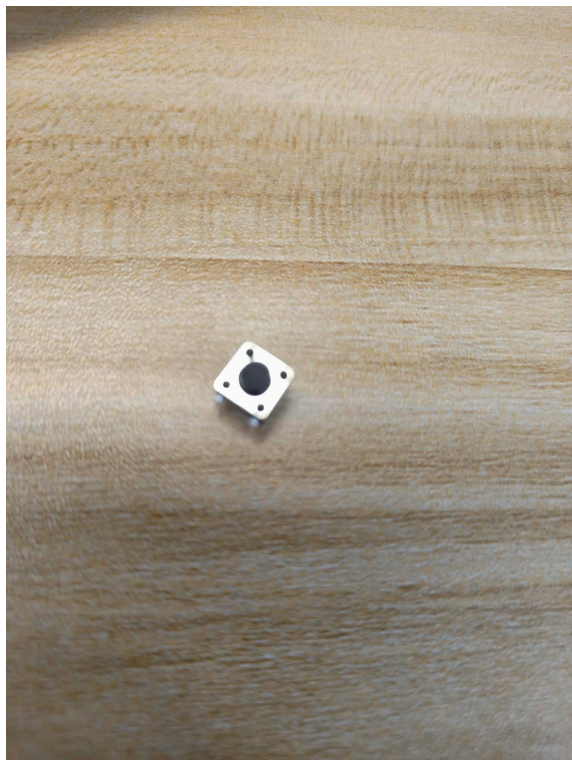
露出金屬針的是公頭 (Male)，有插孔的是母頭 (Female)；線色不會自行決定電壓或功能。所需接頭種類依當週器材表，不是每週都用完三種。商品參考卡上的數量與金額是歷史資料，不是學生應買數量或目前售價。

實物照片：成排導線與接頭全貌 	蝦皮商品參考：公對公：兩端皆為金屬針  【環島科技】40P彩色杜邦線 線長10/20/30CM 一排40根 母頭對母頭 公頭對公頭 公頭對母頭彩色排線 規格：20cm，公對公 購買數量：6排 訂單金額：NT\$150
蝦皮商品參考：公對母：金屬針與插孔各一端  【環島科技】40P彩色杜邦線 線長10/20/30CM 一排40根 母頭對母頭 公頭對公頭 公頭對母頭彩色排線 規格：20cm，公對母 購買數量：6排 訂單金額：NT\$150	蝦皮商品參考：母對母：兩端皆為插孔  【環島科技】40P彩色杜邦線 線長10/20/30CM 一排40根 母頭對母頭 公頭對公頭 公頭對母頭彩色排線 規格：20cm，母對母 購買數量：6排 訂單金額：NT\$150

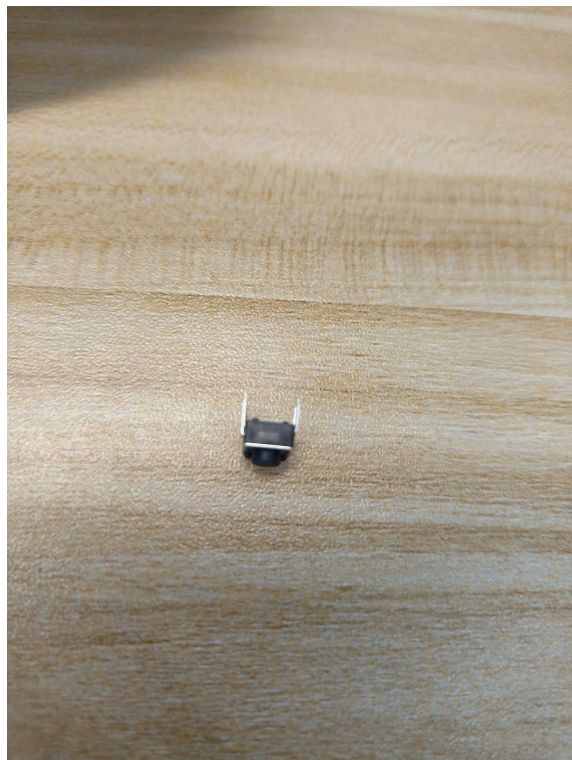
四腳輕觸按鈕 (Tactile Pushbutton)

上方黑色部分是按壓位置，四支金屬腳用來連接電路。照片不能單獨證明哪一對腳常通；先斷電，依 Week 2 的方法辨認。

實物照片：俯視：按鍵與金屬上蓋



實物照片：側面：四支接腳

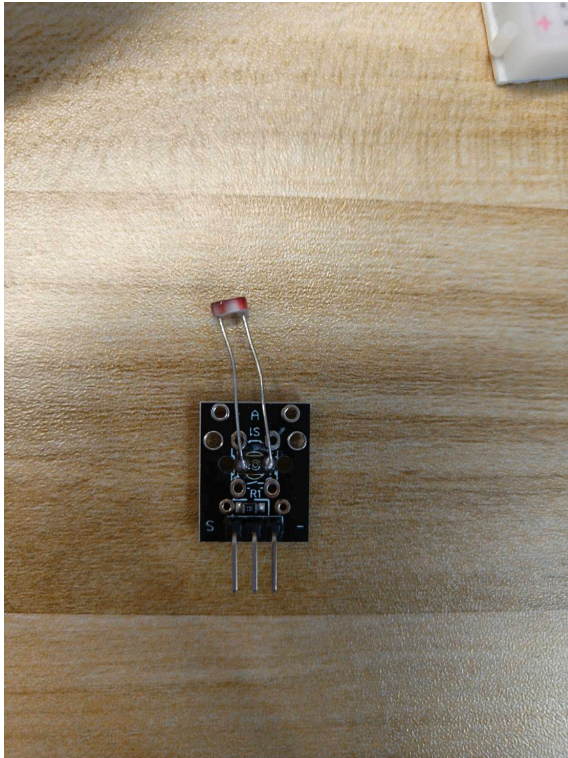


其他留存角度：[歷史接線紀錄](#)：同一組常通接點的量測，不是按下才導通的接法答案。

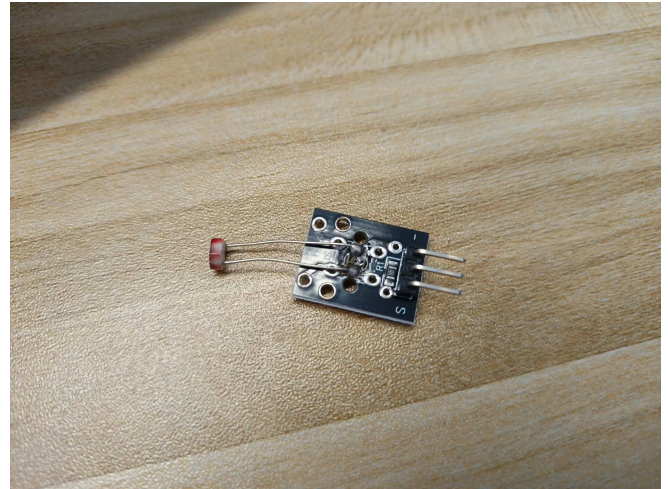
KY-018 光敏電阻模組 (Photoresistor Module)

圓形感光元件、板上固定電阻與三支排針構成模組。S 是訊號標示；元件區的 A、S1、R1 不能直接當成中間排針名稱。接線沿用已確認的 Week 3 紀錄。

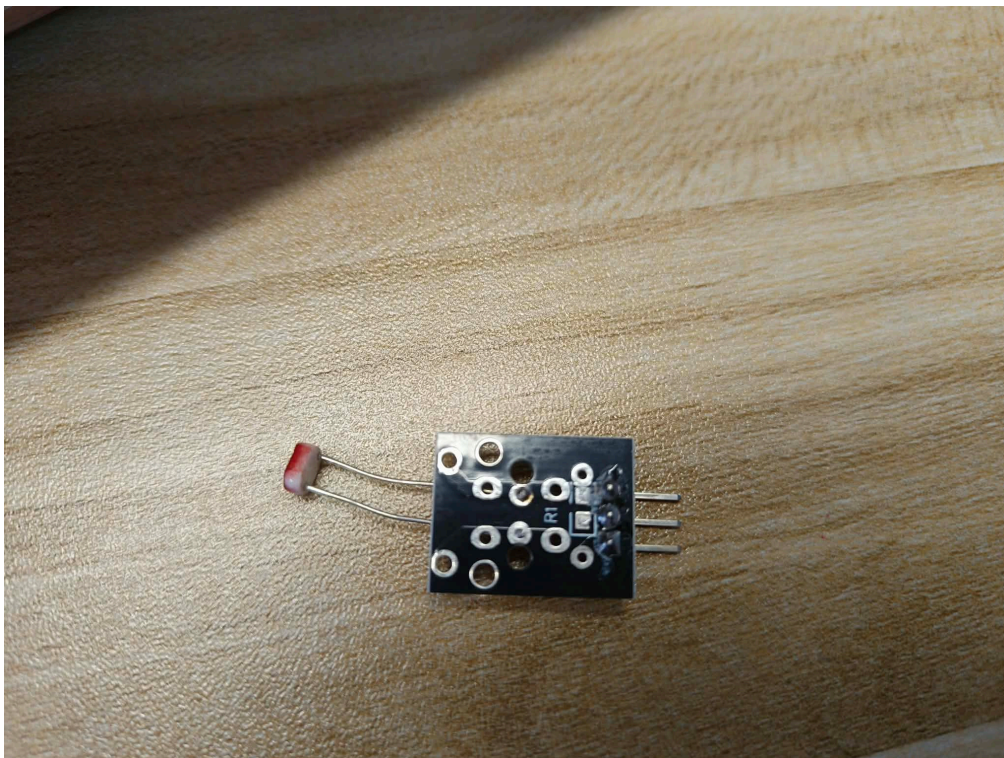
實物照片：正面近照：感光元件、S 與 - 絲印



實物照片：另一元件面角度

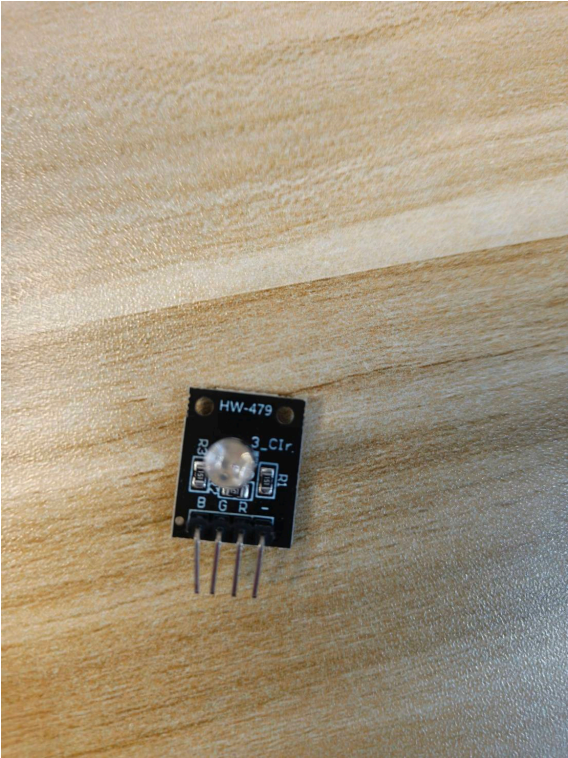
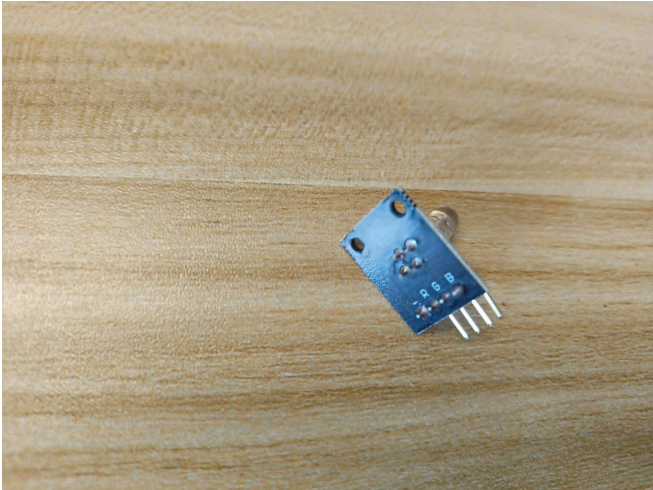


實物照片：焊接面



HW-479 三色發光二極體模組 (RGB LED Module)

訂單稱 KY-016 ；實物 PCB 標示 HW-479 ，前方可見 B 、 G 、 R 、 - 與板上電阻 。共同端、阻值及控制電流仍須核對，不能用八顆燈條取代這個模組。

實物照片：正面：單顆 LED 與 B / G / R / - 標示	實物照片：焊接面
	

其他留存角度：[較早的元件面照片](#)；[失焦補充照](#)；[不供腳位或焊點判讀](#)。

二、先定義Automation Policy

automation (自動反應) 不是「感測值一變就做事」，而是一組可檢查的決策。共同實驗 使用KY-018判斷環境進入較暗條件，RGB由IDLE藍色進入ACTIVE綠色；恢復較亮、超過最大 時間、感測無效、網路長時間中斷或STOP時，回到IDLE或ERROR安全狀態。

開始寫程式前先填[Week 15支援資料](#)，至少包含：

Element	本週共同規則
Trigger	連續3筆有效sample符合dark-enter門檻
Permission	auto mode已啟用、目前IDLE、profile完整
Action	進ACTIVE，RGB顯示綠色並記錄原因
Release	連續3筆有效sample符合light-exit門檻
Maximum duration	ACTIVE最多10秒，超過進ERROR

Element	本週共同規則
Invalid sensing	下一次感測處理進ERROR，不把無效值當0
Network loss	ACTIVE期間MQTT離線超過30秒進ERROR
Physical STOP	最高優先；下一次本機輸入處理進ERROR，不等待network結果
Recovery	查明原因後reset回IDLE；reset不自動重做舊命令

門檻的確切raw值與方向必須來自本人Week 3 profile。沒有共同固定數字，因為模組、ADC、接線與環境不同。

三、Hysteresis與連續取樣

單一threshold附近的noise可能使state快速來回。hysteresis (遲滯) 使用兩個不同門檻：

- DARK_ENTER_RAW：進入dark條件的門檻。
- LIGHT_EXIT_RAW：離開dark條件的門檻。

若「越暗raw越小」，enter值必須小於exit值；若「越暗raw越大」，enter值必須大於exit值。此外連續三筆符合才轉換，避免單一偶發值觸發。這不是任意平均；sample count與兩門檻都必須寫進policy與log。

例如同輪假資料為暗 300 ~ 320、亮 900 ~ 920，可示範 enter=513、exit=706，方向為 DARK_WHEN_RAW_LESS=true。IDLE 連續三筆不大於 513 才開始；ACTIVE 連續三筆不小於 706 才回待機；600 位於兩門檻之間，不觸發轉換。這些數字只用來讀懂規則，實際設定取本組兩段基準之間的不同值，並落在有效範圍內。

四、安全優先順序

程式在同一個loop可能同時看到STOP、sensor change、network command與timeout。固定順序：

1. Physical STOP
2. Invalid sensor / action timeout / network safety timeout
3. Remote stop
4. Reset after fault condition is cleared
5. Manual start (auto mode關閉時)
6. Automatic enter / exit
7. Telemetry與介面更新

高優先動作不可被低優先動作同一loop覆蓋。例如STOP使state進ERROR後，sensor仍然dark也 不得立刻回ACTIVE。**safe state (安全狀態)** 是故障時輸出應進入的明確狀態；共同RGB 實驗為紅色ERROR。學生專題若使用舵機或馬達，安全狀態還須停止PWM / detach / driver disable並受最大動作時間限制，不能只改畫面顏色。

五、由Week 12程式加入Automation

本節逐項列出自動反應需要的程式修改。使用 Week 15 完整程式時，這些內容已包含，不要重貼；若從 Week 12 的副本修改，才依標示位置加入或替換。以下片段的插入或替換位置均有標示。保持 `DRY_RUN=true` 先編譯，profile 未填時不得啟用硬體。

5.1 在設定常數後加入profile與runtime變數

放在 `TELEMETRY_MS` 之後：

```
// 全部值來自本人Week 3校正，不可照抄placeholder。
const bool DARK_WHEN_RAW_LESS = true;
const int DARK_ENTER_RAW = -1;
const int LIGHT_EXIT_RAW = -1;
const int REQUIRED_CONSECUTIVE_SAMPLES = 3;

const unsigned long AUTOMATION_SAMPLE_MS = 500;
const unsigned long MAX_ACTIVE_MS = 10000;
const unsigned long NETWORK_GRACE_MS = 30000;

bool autoMode = false;
bool latestSensorValid = false;
bool simulatedSensorFault = false;
int latestLightRaw = -1;
int darkCount = 0;
int lightCount = 0;
unsigned long lastAutomationSampleAt = 0;
unsigned long activeStartedAt = 0;
unsigned long mqttOfflineSince = 0;
```

5.2 擴充profile檢查

在原 `profileReady()` 後加入：

```
bool automationProfileReady() {
    if (!profileReady()) return false;
    if (DARK_ENTER_RAW < LIGHT_VALID_MIN || DARK_ENTER_RAW > LIGHT_VALID_MAX) return false;
    if (LIGHT_EXIT_RAW < LIGHT_VALID_MIN || LIGHT_EXIT_RAW > LIGHT_VALID_MAX) return false;
    if (DARK_WHEN_RAW_LESS) return DARK_ENTER_RAW < LIGHT_EXIT_RAW;
    return DARK_ENTER_RAW > LIGHT_EXIT_RAW;
}

bool meetsDarkEnter(int raw) {
    return DARK_WHEN_RAW_LESS ? raw <= DARK_ENTER_RAW : raw >= DARK_ENTER_RAW;
}

bool meetsLightExit(int raw) {
    return DARK_WHEN_RAW_LESS ? raw >= LIGHT_EXIT_RAW : raw <= LIGHT_EXIT_RAW;
}
```

5.3 替換enterState()

用下列完整函式替換Week 12原函式，加入ACTIVE起始時間與每次transition log：

```
void enterState(DeviceState next, const char *reason) {
    DeviceState previous = state;
    state = next;
    if (state == DeviceState::IDLE) setRgb(false, false, true);
    if (state == DeviceState::ACTIVE) setRgb(false, true, false);
    if (state == DeviceState::ERROR_STATE) setRgb(true, false, false);
    if (state == DeviceState::ACTIVE && previous != DeviceState::ACTIVE) {
        activeStartedAt = millis();
    }
    if (state != DeviceState::ACTIVE) activeStartedAt = 0;
    Serial.printf("action=state_transition from=%d to=%s reason=%s raw=%d auto=%s\n",
        static_cast<int>(previous), stateName(), reason, latestLightRaw,
        autoMode ? "true" : "false");
}
```

5.4 新增安全錯誤與自動判斷函式

放在readPhysicalInputs()之前：

```
void enterSafetyError(const char *reason) {
    autoMode = false;
    darkCount = 0;
    lightCount = 0;
    enterState(DeviceState::ERROR_STATE, reason);
    publishEvent("safety_error", latestLightRaw, "adc_raw", false, reason);
}

void evaluateNetworkSafety() {
    if (mqttClient.connected()) {
        mqttOfflineSince = 0;
        return;
    }
    if (mqttOfflineSince == 0) mqttOfflineSince = millis();
    if (state == DeviceState::ACTIVE &&
        millis() - mqttOfflineSince >= NETWORK_GRACE_MS) {
        enterSafetyError("network_timeout");
    }
}

void evaluateAutomation() {
    evaluateNetworkSafety();
    if (state == DeviceState::ACTIVE && activeStartedAt != 0 &&
        millis() - activeStartedAt >= MAX_ACTIVE_MS) {
        enterSafetyError("active_timeout");
        return;
    }
    if (DRY_RUN || !automationProfileReady()) return;
    unsigned long now = millis();
    if (now - lastAutomationSampleAt < AUTOMATION_SAMPLE_MS) return;
    lastAutomationSampleAt = now;
    latestLightRaw = analogRead(PIN_LIGHT);
    latestSensorValid = !simulatedSensorFault &&
```

```

        latestLightRaw >= LIGHT_VALID_MIN &&
        latestLightRaw <= LIGHT_VALID_MAX;
Serial.printf("action=automation_sample uptime_ms=%lu raw=%d valid=%s "
              "state=%s auto=%s dark_before=%d light_before=%d\n",
              now, latestLightRaw, latestSensorValid ? "true" : "false",
              stateName(), autoMode ? "true" : "false", darkCount, lightCount);
if (!latestSensorValid) {
    if (state != DeviceState::ERROR_STATE) {
        enterSafetyError(simulatedSensorFault ? "simulated_sensor_invalid" :
                        "sensor_out_of_profile");
    }
    return;
}
if (!autoMode) return;

if (state == DeviceState::IDLE) {
    darkCount = meetsDarkEnter(latestLightRaw) ? darkCount + 1 : 0;
    lightCount = 0;
    if (darkCount >= REQUIRED_CONSECUTIVE_SAMPLES) {
        darkCount = 0;
        enterState(DeviceState::ACTIVE, "automation_dark_confirmed");
        publishEvent("automation_started", latestLightRaw, "adc_raw", true,
                    "three_dark_samples");
    }
} else if (state == DeviceState::ACTIVE) {
    lightCount = meetsLightExit(latestLightRaw) ? lightCount + 1 : 0;
    darkCount = 0;
    if (lightCount >= REQUIRED_CONSECUTIVE_SAMPLES) {
        lightCount = 0;
        enterState(DeviceState::IDLE, "automation_light_confirmed");
        publishEvent("automation_stopped", latestLightRaw, "adc_raw", true,
                    "three_light_samples");
    }
}
}
}

```

5.5 替換command處理

用下列函式替換原executeCommand()：

```

void executeCommand(const String &id, const String &command) {
    int previousIndex = findProcessedCommand(id);
    if (previousIndex >= 0) {
        publishAck(id, processedCommands[previousIndex].result.c_str(),
                  processedCommands[previousIndex].message.c_str());
        return;
    }
    publishAck(id, "accepted", "received by device");

    if (command == "test_sensor_fault") {
        simulatedSensorFault = true;
        latestSensorValid = false;
        enterSafetyError("simulated_sensor_invalid");
        finishCommand(id, "done", "sensor fault test active; safe output applied");
    } else if (command == "clear_sensor_fault_test") {
        simulatedSensorFault = false;
        latestSensorValid = false;
    }
}

```

```

    finishCommand(id, "done",
        "sensor fault test cleared; wait for a fresh valid sample before reset");
} else if (command == "stop") {
    enterSafetyError("remote_stop");
    finishCommand(id, "done", "safe output applied");
} else if (command == "reset") {
    if (!DRY_RUN && stopStablePressed) {
        finishCommand(id, "rejected", "release physical stop before reset");
    } else if (!latestSensorValid && !DRY_RUN) {
        finishCommand(id, "rejected", "sensor must be valid before reset");
    } else {
        autoMode = false;
        enterState(DeviceState::IDLE, "remote_reset");
        finishCommand(id, "done", "idle output applied; auto mode off");
    }
} else if (command == "auto_on") {
    if (state != DeviceState::IDLE || !automationProfileReady() ||
        (!latestSensorValid && !DRY_RUN)) {
        finishCommand(id, "rejected",
            "idle state, valid sensor, and verified profile required");
    } else {
        autoMode = true;
        darkCount = 0; lightCount = 0;
        finishCommand(id, "done", "automation armed");
    }
} else if (command == "auto_off") {
    autoMode = false;
    if (state == DeviceState::ERROR_STATE) {
        finishCommand(id, "done", "automation disabled; error remains latched");
    } else {
        enterState(DeviceState::IDLE, "remote_auto_off");
        finishCommand(id, "done", "automation disabled and idle applied");
    }
} else if (command == "start" && !autoMode &&
    state != DeviceState::ERROR_STATE) {
    enterState(DeviceState::ACTIVE, "remote_manual_start");
    finishCommand(id, "done", "manual active output applied");
} else if (command == "start" && autoMode) {
    finishCommand(id, "rejected", "disable auto mode before manual start");
} else {
    finishCommand(id, "rejected", "command not allowed in current state");
}
}

```

手機基準頁面只有start / stop / reset。測試auto_on與auto_off可用/docs建立命令，test_sensor_fault與clear_sensor_fault_test也只作本週低功率RGB實驗的故障注入。這些命令可用/docs建立，或在自己的前台加入清楚標示的測試控制；仍須帶正確 X-IoT-Key，不得把測試命令未經風險評估直接保留在公開或高功率系統。

5.6 替換physical input並修改loop

STOP永遠先處理；用下列函式替換原readPhysicalInputs()：

```

void readPhysicalInputs() {
    if (DRY_RUN || !profileReady()) return;

```

```

bool stopPressedEvent =
  pressedEvent(PIN_STOP, stopLastRaw, stopStablePressed, stopChangedAt);
bool startPressedEvent =
  pressedEvent(PIN_START, startLastRaw, startStablePressed, startChangedAt);
if (stopStablePressed) {
  if (stopPressedEvent || state != DeviceState::ERROR_STATE) {
    enterSafetyError("physical_stop");
  }
  return;
}
if (startPressedEvent) {
  if (autoMode || state == DeviceState::ERROR_STATE) {
    publishEvent("start_rejected", 1, "pressed", false,
      "manual_start_not_allowed");
  } else {
    enterState(DeviceState::ACTIVE, "physical_manual_start");
    publishEvent("start_pressed", 1, "pressed", true, "physical_input");
  }
}
}
}

```

在loop()的readPhysicalInputs();之後加入：

```

evaluateAutomation();

```

完整loop順序應保持physical input在network reconnect與automation之上；不得用無限while 重連 broker。

另外在 setup() 的啟動訊息把 week=12 改成 week=15；只改這段文字，方便確認上傳的是哪一週，不改 topic。完整程式已包含此修改。

六、分階段驗證Automation

命令要在哪裡送

先讀這段了解按鈕位置，等 6.3 的指定步驟才送命令，現在不要先啟動自動反應。

1. 筆電瀏覽器開 <http://127.0.0.1:8000/docs>，找到綠色 **POST /api/commands** 並展開。
2. 按 **Try it out**。在 x-iot-key 欄位 (HTTP 標頭 X-IoT-Key) 輸入 D 後端視窗設定的同一個臨時 key，勿截圖。
3. 把 Request body 換成以下內容，其中 device_id 改成自己的 .ino 裝置代號：

```

{"device_id":"replace-with-team-device-id","command":"auto_on","parameters":{}}

```

4. 到 6.3 要求送 auto_on 時才按 **Execute**。看到 201、command_id、requested，只代表後端已收件。回手機 **Recent commands** 找同一個 ID，等 done 或實際錯誤結果。
5. 後面要送 auto_off、test_sensor_fault、clear_sensor_fault_test 或 reset 時，只改 JSON 的 command 字串，再按 Execute。一次等前一筆結果出現才送下一筆。
6. 403 先查 key；422 查 JSON / 裝置代號；timeout 查 A、D、E 及 ESP32 連線，不用重按 Execute 掩蓋問題。這些測試只用於本課低功率 RGB，不操作高功率負載。

6.1 Dry run與compile

1. DRY_RUN=true、硬體profile保留實際填值或placeholder。
2. Verify程式，記錄board package、PubSubClient、ArduinoJson與compile結果。
3. 檢查所有新函式只有一份，沒有留下兩個executeCommand()或enterState()。
4. Compile通過不代表門檻方向、安全動作或實體反應正確。

6.2 上電前檢查

1. 拔USB，只保留 Week 12 第八節已確認的 KY-018、START、STOP、RGB 接法；不接SG90 / 蜂鳴器 / 4AA。
2. 核對Week 3有效範圍、dark方向、enter與exit門檻。
3. 確認profile logic：raw越暗越小時enter < exit；越暗越大時enter > exit。
4. 從ESP32沿線到模組、再反向檢查；接回USB前確認3V3、GND與signal。

6.3 Baseline sequence

1. 改DRY_RUN=false，拍照後拔下所有板端杜邦線，只接 USB 後 Verify、Upload。完成後拔 USB、依 Week 12 接線表恢復，接 USB 並開 Monitor 選 115200；開機應IDLE、auto off。
2. 先取得一筆有效KY-018 sample，確保latestSensorValid=true。
3. 送auto_on，ack為done但state仍IDLE，表示只是armed。
4. 使環境符合dark-enter；前兩筆不動作，連續第三筆後進ACTIVE。
5. 在enter與exit門檻中間改變光線，state不應快速來回。
6. 使環境符合light-exit；連續第三筆後回IDLE。
7. 再次進ACTIVE但不恢復光線，10秒後進ERROR，auto off，RGB紅色。
8. 條件恢復且sensor valid後送reset，回IDLE；不自動重新arm。
9. 送test_sensor_fault，確認進ERROR且reason為simulated_sensor_invalid；再送 clear_sensor_fault_test，等下一筆sample使latestSensorValid=true後才送reset。這一步驗證可重現的程式故障路徑，不宣稱實體訊號線已故障。

觀察 Serial 的 action=automation_sample，這是每次自動判斷讀到的資料，不是每兩秒 發布的 telemetry。dark_before / light_before 是處理這筆之前的連續筆數；例如符合暗條件的三行依次為 0、1、2，第三行之後才出現狀態轉換。先讀完本段再測一輪，ACTIVE 最長只有 10 秒；到期後照步驟8復原，不邊倒數邊翻頁。

七、衝突與優先順序測試

1. Auto mode active時按manual START，應拒絕，避免兩個controller同時控制。
2. Auto ACTIVE時按實體STOP，進入ERROR；同一loop的light-exit不得把它改回IDLE。按住STOP時送reset必須rejected，不能因沒有新的按下邊緣而復歸。
3. ERROR中sensor仍dark，不得自動start。
4. ERROR中送auto_on，應rejected。
5. Auto ACTIVE時送remote stop，應ERROR並記remote_stop。
6. 非ERROR狀態送auto_off時應回IDLE並停止自動反應；ERROR狀態只關閉auto mode，仍保持ERROR，必須確認故障排除後另送reset。

所有測試以Serial、Backend event、command ID、database row、手機狀態與實體RGB共同判斷。

八、Fault Injection與Recovery

至少完成下列五項中的三項，其中physical STOP與sensor invalid為固定必測：

8.1 Physical STOP

Auto ACTIVE時按STOP。預期下一次本機輸入處理後RGB為紅色、auto off、event reason為physical_stop；記錄實測最長反應時間，網路同時斷線也不影響本機結果。

8.2 Sensor invalid

共同必測使用test_sensor_fault，程式須在下一個命令處理流程進ERROR、RGB紅色、auto off，並保存simulated_sensor_invalid。送clear_sensor_fault_test只會撤除測試旗標；必須再取得一筆profile內的新sample，才能送reset。這個結果只證明invalid-data處理與recovery路徑，不證明實體斷線偵測。

若教師已用同批KY-018與接線驗證某個安全、可重現的實體故障方式，可另做physical fault test；所有改線先拔USB，並記錄實際raw與reason。不得假設單純拔除signal一定超出profile，也不得為了得到特定數字短接3V3、GND或GPIO。

8.3 Broker / Backend unavailable

Auto ACTIVE後停止broker，觀察MQTT斷線及重連紀錄。這個基準同時有ACTIVE最多10秒及MQTT連續離線30秒兩個限制，所以正常保持ACTIVE時會先由10秒動作期限進ERROR，不能把這次結果寫成「30秒網路timeout已測通過」。期間實體STOP由下一次本機輸入處理生效；網路呼叫可能延後下一輪，須記錄實測反應時間。復原broker可重連，但ERROR不自動reset。

只停止Backend而broker仍在線，不一定造成MQTT斷線；應另外記錄API / 歷史資料 / 前台停止更新，不能由mqtt.connected()推定Backend健康。這兩項故障分開判讀，不為了測30秒而自行放寬安全期限。

8.4 Stuck action timeout

保持dark條件，使ACTIVE不會由light-exit結束。10秒後必須ERROR。若專題使用舵機 / 馬達，另外量測實體動作確實停止，不能只看state文字。

8.5 Malformed / duplicate command

無效JSON不得執行；重複command ID不得重複啟動。不同ID但相同action仍是新命令，需依目前state與policy判斷，而不是全部忽略。

Week 12的RAM cache只涵蓋最近8筆且會在重啟後消失。本項測試先驗證同一次開機的重送，再把「跨重啟仍可能重做」記為限制；有機械或高功率輸出的專題須加入耐久完成紀錄或可安全重做的command語意，不能把QoS 1誤當成exactly-once執行保證。

每個故障要記錄baseline、唯一變因、預測、實際、安全輸出、手機狀態、log、根因、復原 步驟與baseline恢復。完整表在支援資料。

九、Clean Reconstruction

clean reconstruction (乾淨重建) 由沒有使用原開發環境的人，在新資料夾中依repository 文件建立可運作系統。它驗證文件與依賴，不是把原電腦.venv、runtime database與secrets 整包複製。

9.1 原開發者準備

1. 所有必要程式與文件已在Git tracked files中。
2. requirements.txt固定Python依賴；Arduino library與board package版本寫入紀錄。
3. .env.example與device_secrets.example.h只含placeholder。
4. README包含Backend啟動、host test、API與停止方式。
5. git status --short不含秘密；提供要重建的完整commit hash。

9.2 重建者建立全新資料夾

選一個新的空資料夾名稱，不刪除原repository：

```
git clone https://github.com/KennethWYLee/IoT.git iot-week15-rebuild
Set-Location .\iot-week15-rebuild
git checkout <exact-commit-hash>
git status --short
```

正常status為空。若commit尚未push，重建者不能靠原電腦未提交檔案補齊；先記為文件 / 發布 缺漏。

9.3 Backend reconstruction

```
Set-Location .\IOT_Introduction\examples\course_backend
Test-Path .\app.py
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m pytest -q
$env:IOT_OPERATOR_KEY="a-new-local-rebuild-key"
.\.venv\Scripts\python.exe -m uvicorn app:app --host 127.0.0.1 --port 8000
```

第二行必須為 `True` 才執行後面命令。這一段只讓新筆電本機重建，不要求手機能連上。若稍後測手機，先完成本機測試並 `Ctrl+C`，再依 Week 11 的可信任 LAN 設定重啟，不能用 `127.0.0.1` 的本機綁定位址直接要求另一台裝置連線。

新database應由程式建立；不要複製原runtime/iot_course.db。Host test通過後開 `http://127.0.0.1:8000`，檢查manifest、service worker與API。這仍不等於LAN / target test。

9.4 Device reconstruction

1. 依實際紀錄安裝相同esp32 board package、ArduinoJson與PubSubClient。
2. 從[device secrets.example.h](#)建立本機 secrets.h並填重建環境自己的host與臨時密碼。
3. 由profile紀錄填GPIO、ON level、有效範圍與automation門檻；不猜腳位。
4. 先`DRY_RUN=true` compile，再Upload與network dry run。
5. 只有接線檢查完成才`DRY_RUN=false`做physical target test。

9.5 Reconstruction pass condition

另一位重建者能在不取得原.venv、runtime database、secrets.h或口頭隱藏步驟的情況下：

- 執行host tests；
- 啟動Backend並建立新schema；
- 開啟mobile page；
- compile device firmware；
- 知道哪些target test仍需特定實體profile與硬體。

重建失敗時記第一個缺漏、實際錯誤、文件修正與重新測試；不能由原作者直接接手操作後 宣稱文件完整。

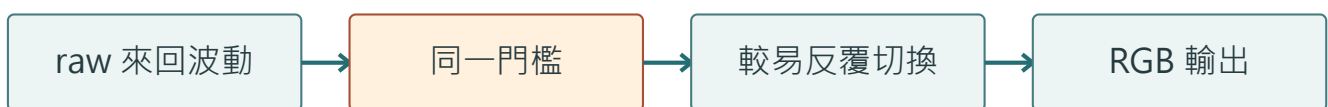
沒有遲滯，邊界附近會怎麼切換？

感測到動作示意；僅作圖上推演，不停止正在使用的服務或取消真實系統的安全檢查。

正常的連接 / 處理



只改標記的地方



箭頭表示資訊處理順序，不是供電或電流路徑。

想一想： 每次取樣間隔相同，三筆確認也保留，只把進入 / 離開改成同一門檻 600。連續三筆 599，再三筆 601，會怎樣？

原理提示： 單一門檻可每三筆就切換；分開門檻時，這些中間值可保持原狀態。

只改圖中標記處，其餘條件保持相同。請指出哪一段仍工作，以及目前證據不能說明什麼。

只改 state，忘了更新輸出呢？

程式狀態與輸出示意；僅作圖上推演，不停止正在使用的服務或取消真實系統的安全檢查。

正常的連接 / 處理



只改標記的地方



箭頭表示資訊處理順序，不是供電或電流路徑。

想一想：只把 state 改為 ERROR 並印字，不執行相應輸出。看到 ERROR 能證明實體已安全嗎？

原理提示：不能。變數、紀錄與輸出是不同步驟；少了輸出命令，腳位可能仍保持舊值。

只改圖中標記處，其餘條件保持相同。請指出哪一段仍工作，以及目前證據不能說明什麼。

十、練習

練習1：啟動與停止使用不同的確認次數

將自動反應改成：IDLE 連續四筆符合暗門檻才進 ACTIVE；ACTIVE 連續兩筆符合亮門檻就回 IDLE。兩種累積次數要分開，不符合目前條件的一筆就清除該次累積。無效資料不是重新計數而已，仍須進 ERROR、關閉 auto。STOP、最大 ACTIVE 時間及網路安全限制沿用基本程式，不延長等待。

事件 reason 要能分辨四筆暗啟動與兩筆亮停止，不能沿用 three 的文字。

預期結果

假設 auto 已啟用、IDLE、資料均有效，且無其他故障；暗是 raw≤513、亮是 raw≥706。這些門檻是教學假資料，不能抄成實物設定。相鄰樣本間隔 500 ms。

依序讀入 raw	預期 state	目前連續次數
300	IDLE	暗 1
310	IDLE	暗 2
600	IDLE	暗 0

依序讀入 raw	預期 state	目前連續次數
300	IDLE	暗 1
310	IDLE	暗 2
305	IDLE	暗 3
315	ACTIVE	切換後清除累積
900	ACTIVE	亮 1
600	ACTIVE	亮 0
910	ACTIVE	亮 1
915	IDLE	切換後清除累積

另做兩項測試：

- 累積暗 3 筆後收到無效資料：ERROR、auto=false，不啟動。
- 累積中按本機 STOP：確認按壓成立後進 ERROR，不等待下一個光線樣本。

測試結果須註明紙上、主機或實物來源；不要用短路或拔動帶電接線製造故障。

練習2：專題優先順序

把自己的專題input、output與failure代入共同priority table。若使用高電流裝置，寫出 driver disable、external power與physical stop如何實現；GPIO LOW文字不能取代電氣設計。

練習3：修一個重建缺漏

重建者選第一個造成停止的文件缺漏，原作者只依重建紀錄修README / example / dependency，由重建者從失敗步驟重新執行並記結果。

十一、繳交內容與完成條件

繳交automation policy、Week 3 profile來源、完整程式差異、hysteresis資料、baseline sequence、衝突矩陣、至少三項fault (含STOP與sensor invalid)、安全輸出與recovery、structured log與 mobile evidence、exact commit clean reconstruction及文件修正紀錄。

- ☐ Trigger、permission、action、release、maximum time、safe state與recovery明確。
- ☐ Enter / exit門檻方向正確，連續三筆與hysteresis實測通過。
- ☐ Physical STOP優先於network、manual與automation，Backend離線也有效。
- ☐ Invalid sensor、timeout與選定network fault進ERROR，無限動作不可能發生。
- ☐ ERROR不自動reset或重做舊命令，recovery前先確認fault已排除。
- ☐ Phone、Backend、database、Serial與physical output對同一事件 / command一致。
- ☐ 另一位學生從新folder與exact commit完成host reconstruction。
- ☐ 新database由schema建立，沒有複製runtime資料、.venv或secret files。
- ☐ Compile、host、network、physical、fault及reconstruction分別標示。
- ☐ Secret scan通過，文件只有placeholder與環境變數名稱。

結束時停用auto mode、使裝置回IDLE，再停止bridge、Backend與broker並拔USB。保留原始 baseline及重建folder供檢查，不使用破壞性命令清除。完整policy、fault、recovery與重建 表格見[Week 15支援資料](#)。

完整基本程式

檔名：week15_automation_device.ino。這是第五節所列修改合併後的完整基本程式。secrets.h 仍使用 Week12 已建立的個人設定；下方範例保留安全佔位值。校正值與接線尚未確認時保持 DRY_RUN=true，不啟用實體輸出。

```
// Generated from the maintained lesson; do not edit this repository copy.
// Save a personal sketch before entering settings or secrets.
#include <Arduino.h>
#include <ArduinoJson.h>
#include <PubSubClient.h>
#include <WiFi.h>
#include "secrets.h"

const char DEVICE_ID[] = "replace-with-team-device-id";
const bool DRY_RUN = true;
const int PIN_START = -1;
const int PIN_STOP = -1;
const int PIN_LIGHT = -1;
const int PIN_RGB_R = -1;
const int PIN_RGB_G = -1;
const int PIN_RGB_B = -1;
const int RGB_ON_LEVEL = -1;
const int LIGHT_VALID_MIN = -1;
const int LIGHT_VALID_MAX = -1;

enum class DeviceState { IDLE, ACTIVE, ERROR_STATE };
```

```

DeviceState state = DeviceState::IDLE;
WiFiClient wifiClient;
PubSubClient mqttClient(wifiClient);

String topicTelemetry, topicEvents, topicPresence, topicCommands, topicAcks;
bool startLastRaw = false, stopLastRaw = false;
bool startStablePressed = false, stopStablePressed = false;
unsigned long startChangedAt = 0, stopChangedAt = 0;
unsigned long lastWifiAttemptAt = 0, lastMqttAttemptAt = 0, lastTelemetryAt = 0;
const unsigned long DEBOUNCE_MS = 35;
const unsigned long WIFI_RETRY_MS = 10000;
const unsigned long MQTT_RETRY_MS = 5000;
const unsigned long TELEMETRY_MS = 2000;

// 全部值来自本人Week 3校正 · 不可照抄placeholder。
const bool DARK_WHEN_RAW_LESS = true;
const int DARK_ENTER_RAW = -1;
const int LIGHT_EXIT_RAW = -1;
const int REQUIRED_CONSECUTIVE_SAMPLES = 3;

const unsigned long AUTOMATION_SAMPLE_MS = 500;
const unsigned long MAX_ACTIVE_MS = 10000;
const unsigned long NETWORK_GRACE_MS = 30000;

bool autoMode = false;
bool latestSensorValid = false;
bool simulatedSensorFault = false;
int latestLightRaw = -1;
int darkCount = 0;
int lightCount = 0;
unsigned long lastAutomationSampleAt = 0;
unsigned long activeStartedAt = 0;
unsigned long mqttOfflineSince = 0;

struct ProcessedCommand {
    String id;
    String result;
    String message;
};
const int PROCESSED_COMMAND_CAPACITY = 8;
ProcessedCommand processedCommands[PROCESSED_COMMAND_CAPACITY];
int nextProcessedCommand = 0;

const char *stateName() {
    switch (state) {
        case DeviceState::IDLE: return "idle";
        case DeviceState::ACTIVE: return "active";
        case DeviceState::ERROR_STATE: return "error";
    }
    return "unknown";
}

bool identifierReady(const char *value) {
    size_t length = strlen(value);
    if (length == 0 || length > 80 || String(value).startsWith("replace-")) return false;
    for (size_t index = 0; index < length; index++) {
        char character = value[index];
        bool allowed = isAlphaNumeric(character) || character == '.' ||
            character == '_' || character == '-';
    }
}

```

```

    if (!allowed) return false;
}
return isAlphaNumeric(value[0]);
}

bool allPinsUnique(const int *pins, size_t count) {
    for (size_t left = 0; left < count; left++)
        for (size_t right = left + 1; right < count; right++)
            if (pins[left] == pins[right]) return false;
    return true;
}

bool profileReady() {
    const int pins[] = {
        PIN_START, PIN_STOP, PIN_LIGHT, PIN_RGB_R, PIN_RGB_G, PIN_RGB_B
    };
    bool pinsReady = PIN_START >= 0 && PIN_STOP >= 0 && PIN_LIGHT >= 0 &&
        PIN_RGB_R >= 0 && PIN_RGB_G >= 0 && PIN_RGB_B >= 0;
    bool range = LIGHT_VALID_MIN >= 0 && LIGHT_VALID_MAX <= 4095 &&
        LIGHT_VALID_MAX > LIGHT_VALID_MIN;
    bool level = RGB_ON_LEVEL == HIGH || RGB_ON_LEVEL == LOW;
    return pinsReady && allPinsUnique(pins, 6) && range && level;
}

bool automationProfileReady() {
    if (!profileReady()) return false;
    if (DARK_ENTER_RAW < LIGHT_VALID_MIN || DARK_ENTER_RAW > LIGHT_VALID_MAX) return false;
    if (LIGHT_EXIT_RAW < LIGHT_VALID_MIN || LIGHT_EXIT_RAW > LIGHT_VALID_MAX) return false;
    if (DARK_WHEN_RAW_LESS) return DARK_ENTER_RAW < LIGHT_EXIT_RAW;
    return DARK_ENTER_RAW > LIGHT_EXIT_RAW;
}

bool meetsDarkEnter(int raw) {
    return DARK_WHEN_RAW_LESS ? raw <= DARK_ENTER_RAW : raw >= DARK_ENTER_RAW;
}

bool meetsLightExit(int raw) {
    return DARK_WHEN_RAW_LESS ? raw >= LIGHT_EXIT_RAW : raw <= LIGHT_EXIT_RAW;
}

int rgbOffLevel() { return RGB_ON_LEVEL == HIGH ? LOW : HIGH; }

void setRgb(bool red, bool green, bool blue) {
    if (DRY_RUN || !profileReady()) return;
    digitalWrite(PIN_RGB_R, red ? RGB_ON_LEVEL : rgbOffLevel());
    digitalWrite(PIN_RGB_G, green ? RGB_ON_LEVEL : rgbOffLevel());
    digitalWrite(PIN_RGB_B, blue ? RGB_ON_LEVEL : rgbOffLevel());
}

void enterState(DeviceState next, const char *reason) {
    DeviceState previous = state;
    state = next;
    if (state == DeviceState::IDLE) setRgb(false, false, true);
    if (state == DeviceState::ACTIVE) setRgb(false, true, false);
    if (state == DeviceState::ERROR_STATE) setRgb(true, false, false);
    if (state == DeviceState::ACTIVE && previous != DeviceState::ACTIVE) {
        activeStartedAt = millis();
    }
    if (state != DeviceState::ACTIVE) activeStartedAt = 0;
}

```

```

    Serial.printf("action=state_transition from=%d to=%s reason=%s raw=%d auto=%s\n",
        static_cast<int>(previous), stateName(), reason, latestLightRaw,
        autoMode ? "true" : "false");
}

void buildTopics() {
    String root = "course/" + String(DEVICE_ID) + "/";
    topicTelemetry = root + "telemetry";
    topicEvents = root + "events";
    topicPresence = root + "presence";
    topicCommands = root + "commands";
    topicAcks = root + "acks";
}

bool publishJson(const String &topic, JsonDocument &document, bool retained = false) {
    if (!mqttClient.connected()) return false;
    String payload;
    serializeJson(document, payload);
    bool sent = mqttClient.publish(topic.c_str(), payload.c_str(), retained);
    Serial.printf("mqtt=publish topic=%s sent=%s payload=%s\n",
        topic.c_str(), sent ? "true" : "false", payload.c_str());
    return sent;
}

void publishPresence(const char *value) {
    JsonDocument doc;
    doc["device_id"] = DEVICE_ID; doc["event_type"] = "presence";
    doc["value"] = value; doc["unit"] = "status"; doc["state"] = stateName();
    doc["valid"] = true; doc["reason"] = "mqtt_session"; doc["uptime_ms"] = millis();
    publishJson(topicPresence, doc, true);
}

void publishEvent(const char *type, int value, const char *unit,
    bool valid, const char *reason) {
    JsonDocument doc;
    doc["device_id"] = DEVICE_ID; doc["event_type"] = type;
    doc["value"] = value; doc["unit"] = unit; doc["state"] = stateName();
    doc["valid"] = valid; doc["reason"] = reason; doc["uptime_ms"] = millis();
    publishJson(topicEvents, doc);
}

void publishAck(const String &id, const char *result, const char *message) {
    JsonDocument doc;
    doc["device_id"] = DEVICE_ID; doc["command_id"] = id;
    doc["result"] = result; doc["message"] = message;
    publishJson(topicAcks, doc);
}

int findProcessedCommand(const String &id) {
    for (int index = 0; index < PROCESSED_COMMAND_CAPACITY; index++) {
        if (processedCommands[index].id == id) return index;
    }
    return -1;
}

void finishCommand(const String &id, const char *result, const char *message) {
    processedCommands[nextProcessedCommand] = {id, result, message};
    nextProcessedCommand = (nextProcessedCommand + 1) % PROCESSED_COMMAND_CAPACITY;
    publishAck(id, result, message);
}

```

```

}

void executeCommand(const String &id, const String &command) {
    int previousIndex = findProcessedCommand(id);
    if (previousIndex >= 0) {
        publishAck(id, processedCommands[previousIndex].result.c_str(),
            processedCommands[previousIndex].message.c_str());
        return;
    }
    publishAck(id, "accepted", "received by device");

    if (command == "test_sensor_fault") {
        simulatedSensorFault = true;
        latestSensorValid = false;
        enterSafetyError("simulated_sensor_invalid");
        finishCommand(id, "done", "sensor fault test active; safe output applied");
    } else if (command == "clear_sensor_fault_test") {
        simulatedSensorFault = false;
        latestSensorValid = false;
        finishCommand(id, "done",
            "sensor fault test cleared; wait for a fresh valid sample before reset");
    } else if (command == "stop") {
        enterSafetyError("remote_stop");
        finishCommand(id, "done", "safe output applied");
    } else if (command == "reset") {
        if (!DRY_RUN && stopStablePressed) {
            finishCommand(id, "rejected", "release physical stop before reset");
        } else if (!latestSensorValid && !DRY_RUN) {
            finishCommand(id, "rejected", "sensor must be valid before reset");
        } else {
            autoMode = false;
            enterState(DeviceState::IDLE, "remote_reset");
            finishCommand(id, "done", "idle output applied; auto mode off");
        }
    } else if (command == "auto_on") {
        if (state != DeviceState::IDLE || !automationProfileReady() ||
            (!latestSensorValid && !DRY_RUN)) {
            finishCommand(id, "rejected",
                "idle state, valid sensor, and verified profile required");
        } else {
            autoMode = true;
            darkCount = 0; lightCount = 0;
            finishCommand(id, "done", "automation armed");
        }
    } else if (command == "auto_off") {
        autoMode = false;
        if (state == DeviceState::ERROR_STATE) {
            finishCommand(id, "done", "automation disabled; error remains latched");
        } else {
            enterState(DeviceState::IDLE, "remote_auto_off");
            finishCommand(id, "done", "automation disabled and idle applied");
        }
    } else if (command == "start" && !autoMode &&
        state != DeviceState::ERROR_STATE) {
        enterState(DeviceState::ACTIVE, "remote_manual_start");
        finishCommand(id, "done", "manual active output applied");
    } else if (command == "start" && autoMode) {
        finishCommand(id, "rejected", "disable auto mode before manual start");
    } else {

```

```

        finishCommand(id, "rejected", "command not allowed in current state");
    }
}

void onMqttMessage(char *topic, byte *payload, unsigned int length) {
    if (String(topic) != topicCommands) return;
    if (length == 0 || length >= 768) {
        Serial.printf("mqtt=reject reason=payload_length length=%u\n", length); return;
    }
    char buffer[768];
    memcpy(buffer, payload, length); buffer[length] = '\0';
    JsonDocument doc;
    DeserializationError error = deserializeJson(doc, buffer);
    if (error) {
        Serial.printf("mqtt=reject reason=json detail=%s\n", error.c_str()); return;
    }
    String target = doc["device_id"] | "";
    String id = doc["command_id"] | "";
    String command = doc["command"] | "";
    if (target != DEVICE_ID || id.length() == 0 || command.length() == 0) {
        Serial.println("mqtt=reject reason=identity_or_field"); return;
    }
    executeCommand(id, command);
}

void requestWifi() {
    if (WiFi.status() == WL_CONNECTED) return;
    unsigned long now = millis();
    if (lastWifiAttemptAt && now - lastWifiAttemptAt < WIFI_RETRY_MS) return;
    lastWifiAttemptAt = now; WiFi.disconnect(); WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    Serial.println("wifi=connecting");
}

void requestMqtt() {
    if (WiFi.status() != WL_CONNECTED || mqttClient.connected()) return;
    unsigned long now = millis();
    if (lastMqttAttemptAt && now - lastMqttAttemptAt < MQTT_RETRY_MS) return;
    lastMqttAttemptAt = now;

    JsonDocument willDoc;
    willDoc["device_id"] = DEVICE_ID; willDoc["event_type"] = "presence";
    willDoc["value"] = "offline"; willDoc["unit"] = "status";
    willDoc["state"] = stateName(); willDoc["valid"] = true;
    willDoc["reason"] = "last_will";
    String willPayload; serializeJson(willDoc, willPayload);

    bool connected = mqttClient.connect(
        DEVICE_ID, MQTT_USERNAME, MQTT_PASSWORD,
        topicPresence.c_str(), 1, true, willPayload.c_str());
    Serial.printf("mqtt=connect connected=%s state=%d\n",
        connected ? "true" : "false", mqttClient.state());
    if (connected) {
        mqttClient.subscribe(topicCommands.c_str(), 1);
        publishPresence("online");
    }
}

bool pressedEvent(int pin, bool &lastRaw, bool &stablePressed,
    unsigned long &changedAt) {

```

```

    bool pressed = digitalRead(pin) == LOW;
    unsigned long now = millis();
    if (pressed != lastRaw) {
        lastRaw = pressed;
        changedAt = now;
    }
    if (now - changedAt >= DEBOUNCE_MS && pressed != stablePressed) {
        stablePressed = pressed;
        return stablePressed;
    }
    return false;
}

void enterSafetyError(const char *reason) {
    autoMode = false;
    darkCount = 0;
    lightCount = 0;
    enterState(DeviceState::ERROR_STATE, reason);
    publishEvent("safety_error", latestLightRaw, "adc_raw", false, reason);
}

void evaluateNetworkSafety() {
    if (mqttClient.connected()) {
        mqttOfflineSince = 0;
        return;
    }
    if (mqttOfflineSince == 0) mqttOfflineSince = millis();
    if (state == DeviceState::ACTIVE &&
        millis() - mqttOfflineSince >= NETWORK_GRACE_MS) {
        enterSafetyError("network_timeout");
    }
}

void evaluateAutomation() {
    evaluateNetworkSafety();
    if (state == DeviceState::ACTIVE && activeStartedAt != 0 &&
        millis() - activeStartedAt >= MAX_ACTIVE_MS) {
        enterSafetyError("active_timeout");
        return;
    }
    if (DRY_RUN || !automationProfileReady()) return;
    unsigned long now = millis();
    if (now - lastAutomationSampleAt < AUTOMATION_SAMPLE_MS) return;
    lastAutomationSampleAt = now;
    latestLightRaw = analogRead(PIN_LIGHT);
    latestSensorValid = !simulatedSensorFault &&
        latestLightRaw >= LIGHT_VALID_MIN &&
        latestLightRaw <= LIGHT_VALID_MAX;
    Serial.printf("action=automation_sample uptime_ms=%lu raw=%d valid=%s "
        "state=%s auto=%s dark_before=%d light_before=%d\n",
        now, latestLightRaw, latestSensorValid ? "true" : "false",
        stateName(), autoMode ? "true" : "false", darkCount, lightCount);
    if (!latestSensorValid) {
        if (state != DeviceState::ERROR_STATE) {
            enterSafetyError(simulatedSensorFault ? "simulated_sensor_invalid" :
                "sensor_out_of_profile");
        }
        return;
    }
}

```

```

if (!autoMode) return;

if (state == DeviceState::IDLE) {
    darkCount = meetsDarkEnter(latestLightRaw) ? darkCount + 1 : 0;
    lightCount = 0;
    if (darkCount >= REQUIRED_CONSECUTIVE_SAMPLES) {
        darkCount = 0;
        enterState(DeviceState::ACTIVE, "automation_dark_confirmed");
        publishEvent("automation_started", latestLightRaw, "adc_raw", true,
            "three_dark_samples");
    }
} else if (state == DeviceState::ACTIVE) {
    lightCount = meetsLightExit(latestLightRaw) ? lightCount + 1 : 0;
    darkCount = 0;
    if (lightCount >= REQUIRED_CONSECUTIVE_SAMPLES) {
        lightCount = 0;
        enterState(DeviceState::IDLE, "automation_light_confirmed");
        publishEvent("automation_stopped", latestLightRaw, "adc_raw", true,
            "three_light_samples");
    }
}
}

void readPhysicalInputs() {
    if (DRY_RUN || !profileReady()) return;
    bool stopPressedEvent =
        pressedEvent(PIN_STOP, stopLastRaw, stopStablePressed, stopChangedAt);
    bool startPressedEvent =
        pressedEvent(PIN_START, startLastRaw, startStablePressed, startChangedAt);
    if (stopStablePressed) {
        if (stopPressedEvent || state != DeviceState::ERROR_STATE) {
            enterSafetyError("physical_stop");
        }
        return;
    }
    if (startPressedEvent) {
        if (autoMode || state == DeviceState::ERROR_STATE) {
            publishEvent("start_rejected", 1, "pressed", false,
                "manual_start_not_allowed");
        } else {
            enterState(DeviceState::ACTIVE, "physical_manual_start");
            publishEvent("start_pressed", 1, "pressed", true, "physical_input");
        }
    }
}

void publishTelemetryIfDue() {
    if (DRY_RUN || !profileReady() || !mqttClient.connected()) return;
    unsigned long now = millis();
    if (now - lastTelemetryAt < TELEMETRY_MS) return;
    lastTelemetryAt = now;
    int raw = analogRead(PIN_LIGHT);
    bool valid = raw >= LIGHT_VALID_MIN && raw <= LIGHT_VALID_MAX;
    JsonDocument doc;
    doc["device_id"] = DEVICE_ID; doc["event_type"] = "light_sample";
    doc["value"] = raw; doc["unit"] = "adc_raw"; doc["state"] = stateName();
    doc["valid"] = valid;
    doc["reason"] = valid ? "within_profile" : "out_of_profile";
    doc["uptime_ms"] = now;
}

```

```

    publishJson(topicTelemetry, doc);
}

void setup() {
    Serial.begin(115200); delay(500); buildTopics();
    if (!identifierReady(DEVICE_ID)) {
        Serial.println("fatal=device_id_missing_or_invalid"); return;
    }
    if (!DRY_RUN && !profileReady()) {
        Serial.println("fatal=hardware_profile_incomplete"); return;
    }
    if (!DRY_RUN) {
        pinMode(PIN_START, INPUT_PULLUP); pinMode(PIN_STOP, INPUT_PULLUP);
        pinMode(PIN_RGB_R, OUTPUT); pinMode(PIN_RGB_G, OUTPUT); pinMode(PIN_RGB_B, OUTPUT);
        enterState(DeviceState::IDLE, "boot");
    }
    WiFi.mode(WIFI_STA);
    mqttClient.setServer(MQTT_HOST, MQTT_PORT);
    mqttClient.setCallback(onMqttMessage);
    mqttClient.setBufferSize(768);
    mqttClient.setKeepAlive(20);
    mqttClient.setSocketTimeout(1);
    requestWifi();
    Serial.printf("week=15 device=%s topic=%s mode=%s\n",
        DEVICE_ID, topicCommands.c_str(), DRY_RUN ? "dry_run" : "hardware");
}

void loop() {
    readPhysicalInputs();
    evaluateAutomation(); // broker離線時也先處理本機STOP
    requestWifi();
    requestMqtt();
    if (mqttClient.connected()) mqttClient.loop();
    publishTelemetryIfDue();
    delay(5);
}

```

準備、紀錄表與延伸參考

一、Automation Policy表

Policy element	定義	可觀察輸入	State前置條件	動作 / 輸出	Log / event	驗收方式
Dark enter			IDLE + auto on	ACTIVE		
Light exit			ACTIVE	IDLE		
Invalid sensor			任何	ERROR		
Active timeout			ACTIVE	ERROR		
Network timeout			ACTIVE	ERROR		

Policy element	定義	可觀察輸入	State前置條件	動作 / 輸出	Log / event	驗收方式
Physical STOP			任何	ERROR		
Remote stop			任何	ERROR		
Reset		fault已排除	IDLE + auto off			

二、Profile與Hysteresis紀錄

```

Week 3 profile commit / 紀錄：
PIN_LIGHT：
LIGHT_VALID_MIN：
LIGHT_VALID_MAX：
DARK_WHEN_RAW_LESS：true / false
DARK_ENTER_RAW：
LIGHT_EXIT_RAW：
Enter / exit關係檢查：
REQUIRED_CONSECUTIVE_SAMPLES：
AUTOMATION_SAMPLE_MS：
MAX_ACTIVE_MS：
NETWORK_GRACE_MS：

```

門檻附近觀察

Sample	Raw	Valid	Enter條件	Exit條件	dark count	light count	State	是否符合預測
1								
2								
3								
4								
5								
6								

完成條件：在enter與exit之間的小幅變化不造成state chatter；連續sample尚未達指定數時不 提早轉換。

三、Priority與Conflict Matrix

同時條件	較高優先	預期最終state	輸出	低優先動作是否執行	實際證據
Physical STOP + dark enter	STOP	ERROR	Safe	否	

同時條件	較高優先	預期最終state	輸出	低優先動作是否執行	實際證據
Sensor invalid + remote start	invalid sensor	ERROR	Safe	否	
Active timeout + light exit	timeout	ERROR	Safe	否	
Auto mode + manual start	auto ownership	不變 / reject	不變	否	
ERROR + auto_on	ERROR latch	ERROR	Safe	否	
Remote stop + telemetry	stop	ERROR	Safe	telemetry可後送	
Network offline + physical STOP	STOP	ERROR	Safe	不等待network	

若測試結果與priority table不同，先修程式順序或state guard，不修改表格迎合程式錯誤。

四、Baseline Test紀錄

階段	開始state	操作 / input	預期	實際state / RGB	Event / command ID	Pass
1	boot	有效sample	IDLE、auto off			
2	IDLE	auto_on	armed、仍IDLE			
3	IDLE	dark sample 1 / 2	不轉換			
4	IDLE	dark sample 3	ACTIVE			
5	ACTIVE	hysteresis區間	維持ACTIVE			
6	ACTIVE	light sample 1 / 2	不轉換			
7	ACTIVE	light sample 3	IDLE			
8	ACTIVE	超過max duration	ERROR + auto off			
9	ERROR	fault清除 + reset	IDLE + auto off			

五、Fault Injection與Recovery表

Fault	Baseline	唯一變因	預期安全 state	實體結果	Mobile	Structure d log	Databas e	Recovery	Baseline 再驗證
Physical STOP			ERROR						
Sensor invalid (程式注入)		test_sen sor_fa ult	ERROR					清除測試 旗標、取 得新有效 sample、 reset	
Sensor invalid (核准實 體測試， 如有)			ERROR						
Broker unava lable			ERROR ; 核對是 ACTIVE 超時或離 線寬限期 先到						
Backend unava lable			依policy						
Active timeout			ERROR						
Malform ed comman d			state unchang ed						
Duplicat e comman d			no repeated action						

Recovery必須回答

1. Fault是否仍存在？用什麼證據判定？
2. 哪個輸出已進safe state？是實體測試還是只看log？
3. Reset前需要人工檢查什麼？
4. Reset會回到哪個state？Auto mode是否保持關閉？
5. 舊command會不會重播？
6. Recovery後哪一個baseline test重新通過？

六、高功率 / 機械輸出專題附加檢查

共同RGB實驗是低功率可視輸出。專題若使用SG90、馬達、泵、電磁閥或其他負載，另外填：

項目	專題設計	實測證據	未完成時的限制
External power voltage / current			
Common ground			
Driver / isolation			
Startup current / voltage drop			
Driver disable / detach			
Maximum action duration			
Physical emergency stop			
Jam / stuck detection			
Power removal recovery			

GPIO不得直接供應高電流負載。「畫面顯示STOP」不是實體driver已停止的證據。

七、Clean Reconstruction Checklist

原作者交付前

- ☐ 提供exact commit hash，且必要commit已push到可取得remote。
- ☐ `git status --short`已檢查，不依賴untracked必要檔。
- ☐ Backend dependency固定於requirements.txt。
- ☐ Board package與Arduino libraries版本已記錄。
- ☐ .env.example及device secrets example只有placeholder。
- ☐ .gitignore排除.venv、runtime、cache及秘密。
- ☐ README寫明setup、host test、run、stop、API、限制。
- ☐ Hardware profile、接線與安全停止有紀錄。

重建者不得取得

- 原開發者.venv或全域Python site-packages。
- 原runtime/iot_course.db。
- 原secrets.h、broker password file或operator key。
- 原開發者未寫入文件的口頭步驟。
- 原開發者已開啟的Backend / broker程序。

重建步驟紀錄

Step	Command / document section	Expected	Actual	Pass / blocked	First missing fact	Fix commit	Retest
1	clone exact commit	clean status					
2	create venv	local environment					
3	install requirements	no error					
4	pytest	all pass					
5	start Backend	new DB schema					
6	host POST / query	201 / history					

Step	Command / document section	Expected	Actual	Pass / blocked	First missing fact	Fix commit	Retest
7	mobile localhost	page / WS					
8	install Arduino deps	exact versions					
9	device dry-run compile	build success					
10	target profile	documented / pending					

八、Reconstruction Failure分類

分類	例子	修正位置	不可採用的繞過方式
Missing dependency	import error	requirements / version docs	複製原.venv
Missing secret name	不知env var	.env.example / README	提交真實key
Missing command	不知run path	README	原作者代為啟動
Missing schema step	DB table不存在	app init / README	複製原database
Missing frontend asset	404 manifest/sw	tracked files / routes	使用原瀏覽器cache
Missing hardware profile	GPIO不明	Lab Note / profile	猜網路pinout
Environment mismatch	version API不同	version record	靜默升降版
Undocumented external state	broker已預先執行	runbook	使用原程序

九、Secret Scan與資料清理

- ☐ git diff與staged diff沒有SSID、password、key、token。
- ☐ Screenshot、terminal transcript與log沒有command header秘密。
- ☐ .env.example及.h example全部是placeholder。
- ☐ Device ID不含個資。
- ☐ Runtime database與broker password file不在Git。
- ☐ Reconstruction使用全新臨時credential。
- ☐ 若秘密曾進Git，已立即停止分享、撤銷 / 更換，並依repository管理流程處理。

十、Lab Notebook模板

日期 / 組別 / baseline commit :

Automation policy :

- trigger / release :
- validity :
- action / max duration :
- safe state :
- priority :
- recovery :

Hardware profile evidence :

- sensor :
- RGB / output :
- physical STOP :

Baseline :

- compile :
- upload :
- physical sequence :
- Backend / DB / mobile :

Faults :

- 1.
- 2.
- 3.

Reconstruction :

- rebuilder :
- exact commit :
- host pass :
- device compile :
- first missing fact :
- fix / retest :

Unverified layers :

十一、延伸實作

延伸A : Cooldown

完成一次ACTIVE後加入短cooldown，期間拒絕重新start並顯示剩餘條件。Physical STOP不受 cooldown限制。

延伸B : Manual override lease

手動override必須有到期時間，超時回safe state。記錄owner、start、expiry與terminal reason；不得建立永久隱藏override。

延伸C : Boot safety

在network、sensor與profile尚未ready時強制safe output，逐一測試開機按住START、broker 離線、sensor 缺失及brownout後restart。

延伸D : Automated reconstruction test

建立host-only script依序建立venv、安裝、pytest、啟動Backend及API smoke test。Script不得 寫入真實secret，也不能聲稱涵蓋target hardware。

十二、Repository參考

- [Course Backend README](#)
- [Backend environment example](#)
- [Device secrets example](#)
- [Backend host tests](#)
- [Week 12 MQTT主教材](#)
- [Week 14 Mobile主教材](#)